

ARCH-1R/T v3

Operational Dual / Symbolic Reachability Framework

Generation asks what can be made. Realization asks what was reached.

Draft Review v3.0 · Formal Core, Glyph Register, Obstruction Taxonomy, and Worked Examples

Status: This v3 draft preserves the original ARCH-1 foundation and incorporates the external collision review: R/T is the operational dual of ARCH-1; ρ is bound as an orbit through generated space; β -loss/terminal-obstruction is primitive.

□

Contents

- 1. Executive Summary
- 2. What v3 Changes
- 3. Formal Core
- 4. Symbol and Glyph Register
- 5. The Dual- ρ Correction
- 6. Terminal-Obstruction
- 7. Obstruction Taxonomy
- 8. Ternary Reachability
- 9. Dependency Propagation
- 10. UOR Relational Instantiation
- 11. UAR-Agnostic Operational Mapping
- 12. Worked Examples
- 13. Implementation Model
- 14. Falsification and Review Criteria
- 15. Open Questions
- 16. Compact Seals

□

1. Executive Summary

ARCH-1R/T v3 is a reachability and coherence framework built as the operational dual of the original ARCH-1 generative canon. The original ARCH-1 answers the generative question: given a seed β and a constructor Ω , what structured possibility space can be recursively generated? ARCH-1R/T answers the realization question: once that space exists, what is traversed, what is obstructed, what remains latent, what is observed, and how far prediction diverges from reality?

The key v3 refinements are structural rather than decorative. First, traversal is no longer described as an arbitrary scheduler acting on G ; it is the orbit of the canonical engine ρ through G . Second, obstruction is no longer modeled only as a downstream block over traversed states. A primitive pre-generative obstruction is added: terminal-obstruction, or β -loss. If the seed is withdrawn, the generated space never forms. This distinguishes path failure from origin failure.

ARCH-1: $\Sigma := \mu S.(\beta \vee \Omega(S, S))$

Generated: $G := \text{Closure}\Omega(\beta)$

Traversed: $T := \text{orbit}(\rho, G)$

Realized: $R := T \setminus O^*$

Classified: $\tau : T \rightarrow \{+, 0, -\}$

Validated: $\Delta := d(R, Q)$

2. What v3 Changes

Change	Before v3	v3 Revision	Why it matters
Traversal	$T := \rho(G)$	$T := \text{orbit}(\rho, G)$	Binds traversal to the canonical ARCH-1 engine instead of generic scheduling.
Obstruction $O \subseteq T$		$O\beta, O\Omega, O\rho, OT, OD$	Distinguishes seed loss, construction failure, traversal failure, path block, and dependency cascade.
β -loss	Implicit or missing	$\beta = \emptyset \Rightarrow G = T = R = \emptyset$	Models amnesia / terminal loss as pre-generation, not path obstruction.
Latency	Runtime label	τ state and possible open-orbit signature	Keeps 0 distinct from failure and connects potential to traversal.
Duality	Extension language	Operational dual of ARCH-1	Shows R/T extends rather than competes with the original canon.

3. Formal Core

3.1 Original Generative Core

The original ARCH-1 core remains unchanged. It is the kernel of the entire v3 document.

$\Sigma := \mu S.(\beta \vee \Omega(S, S))$

$G := \text{Closure}\Omega(\beta)$

Definitions:

- β : terminal anchor or seed.
- Ω : binary constructor.
- μS : least fixed point or recursive closure.
- G : generated possibility space.

Alignment rule: v3 must not redefine β , Ω , or μ . All later layers operate after, around, or before the generated space without replacing the original recursive construction.

3.2 Operational Dual

ARCH-1R/T begins where ARCH-1 leaves off. If ARCH-1 names what can be made, R/T names what was reached.

$T := \text{orbit}(\rho, G)$

$R := T \setminus O^*$

$\Delta := d(R, Q)$

Here Q denotes observed realization and Δ denotes the mismatch between predicted realization and observation.

3.3 Full v3 Equation

If $\beta \neq \emptyset$:

```

G := ClosureΩ(β)
T := orbit(ρ, G)
O* := closureD(Oβ ∪ OΩ ∪ Oρ ∪ OT ∪ OD)
R := T \ O*
τ : T → {+, 0, -}
Δ := d(R, Q)

```

If $\beta = \emptyset$:

```

G = T = R = ∅

```

4. Symbol and Glyph Register

Symbol	Plain meaning	Operational role	v3 note
β	Seed / terminal anchor	Begins recursive closure	Loss of β is terminal-obstruction.

Ω	Binary constructor	Generates structure recursively	Preserved from original ARCH-1.
μ	Least fixed point	Closes recursive construction	Original theorem layer.
G	Generated space	All structurally generated possibilities	Not yet traversed or classified.
ρ	Engine / walk	Activates and traverses	Read inward as activation and across as orbit.
T	Traversed space	Orbit of ρ through G	Not arbitrary scheduler output.
O*	Propagated obstruction	All effective obstruction after closure	Includes terminal, generator, traversal, path, dependency.
R	Realized / realizable space	Traversed space minus effective obstruction	Target of validation.
τ	Ternary classifier	Classifies +, 0, -	Keeps latent distinct from blocked.
Q	Observed realization	Empirical surface	What actually happened.
Δ	Coherence gap	Distance between R and Q	Falsification surface.
⚔	neg / Swordsman	One involution of ρ	Glyph form of dual engine.
🧙	bnot / Mage	Second involution of ρ	Glyph form of dual engine.
👁	Aletheia	Medium / carrier at ρ -layer	Not backward/forward reachability.
🌀	Lethe	Substrate / unretrievable depth	β -loss / amnesia layer.

5. The Dual- ρ Correction

Earlier R/T drafts risked weakening the canonical engine by describing ρ as any scheduler, planner, execution policy, proof search, or governance route. That made the framework operationally flexible but mathematically loose. v3 corrects this by preserving ρ as the canonical engine and reading it at two scopes.

$\rho := \text{inv}_1 \oplus \text{inv}_2 \mapsto \text{generator}$

$\rho_{\text{inward}} \equiv \rho_{\text{across}}$

$T := \text{orbit}(\rho, G)$

The inward reading activates construction; the across reading walks the generated space. This keeps traversal anchored to the original ARCH-1 engine rather than substituting a generic runtime policy.

Scope	Reading of ρ	Question answered	Example
Inward	Activation engine	How does the constructor become generative?	$\text{neg} \oplus \text{bnot} \mapsto \text{succ.}$
Across	Orbit / walk	Which generated possibilities are crossed?	$\text{orbit}(\rho, G).$

6. Terminal-Obstruction

The most important v3 addition is terminal-obstruction. In prior drafts, obstruction occurred after traversal: paths existed, but some were blocked. Terminal-obstruction occurs before generation: the seed itself is missing, withdrawn, destroyed, or unrecoverable.

$O\beta : \beta \mapsto \emptyset$

$\beta = \emptyset \Rightarrow \text{Closure}\Omega(\beta) = \emptyset$

$\beta = \emptyset \Rightarrow G = T = R = \emptyset$

Principle: Amnesia is not a block. It is the seed withdrawn.

This is not a semantic flourish. It is a formal distinction between two different failure depths. Path obstruction says a traversed state is unavailable. Terminal obstruction says there is no origin from which the generated state space can be formed.

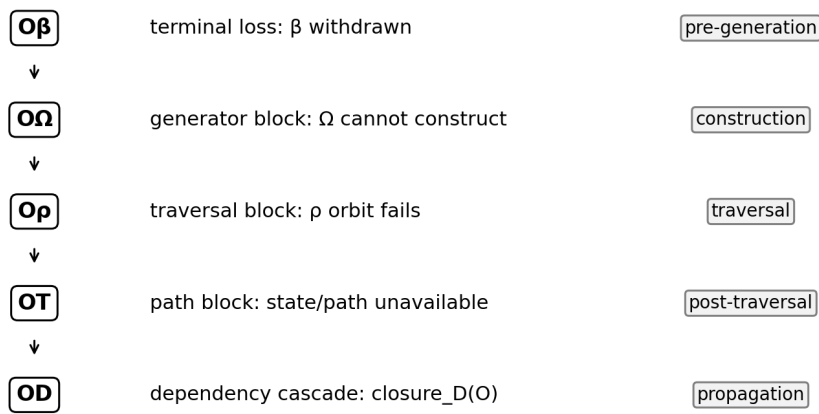
Failure depth	Formal shape	Plain language	Example
Path failure	$x \in OT \subseteq T$	A route exists but is blocked.	Permission denied on an action.
Cascade failure	$O^* = \text{closureD}(O)$	One block propagates through dependencies.	Service dependency failure.
Origin failure	$\beta = \emptyset$	The seed is withdrawn; no closure begins.	Amnesia / unrecoverable terminal loss.

7. Obstruction Taxonomy

v3 separates obstruction by where it enters the architecture.

Class	Location	Equation / trigger	Meaning
O β	Before generation	$\beta \mapsto \emptyset$	Terminal loss: no seed, no closure.
O Ω	During construction	Ω unavailable or invalid	The constructor cannot generate some structure.
O ρ	During traversal	ρ orbit interrupted	The engine cannot walk the generated space.
OT	After traversal	$OT \subseteq T$	Specific paths or states are blocked.
OD	Dependency layer	$\text{closureD}(O)$	Obstruction propagates through dependencies.

$$O^* := \text{closureD}(O\beta \cup O\Omega \cup O\rho \cup OT \cup OD)$$



Read downward by depth: origin → construction → traversal → path → dependency propagation.

8. Ternary Reachability

v3 retains ternary classification because it solves a practical and conceptual problem: not-yet is not impossible. The classifier is placed after traversal to preserve the original generator.

$$\tau : T \rightarrow \{+, 0, -\}$$

State Name	Meaning	Typical transition
+	Reached / active Realizable under current conditions.	$+ \rightarrow 0$ when certainty or availability degrades.
0	Latent / pending Neither realized nor blocked.	$0 \rightarrow +$ by resolution; $0 \rightarrow -$ by obstruction.
-	Obstructed Blocked by active constraint.	$- \rightarrow 0$ by relief; $- \rightarrow +$ by bypass/restoration.

$$T = T+ \cup T0 \cup T-$$

$$T+ \cap T0 = T+ \cap T- = T0 \cap T- = \emptyset$$

$$0 \neq -$$

The speculative v3 insight is that latency may eventually be expressible as an open orbit: a state in which the ρ -walk has not closed. This is not yet established, but it is a promising research target because it would make latency internal to the algebra rather than a runtime annotation.

9. Dependency Propagation

Dependency propagation is where the ternary layer becomes operational. v3 distinguishes hard and soft dependencies.

Hard dependency: $x- \rightarrow y-$

Soft dependency: $x- \rightarrow y0$

A hard dependency propagates obstruction. A soft dependency propagates latency. Thus the neutral law is lifted from single-state classification to cascade behavior.

Dependency type	Propagation rule	Operational reading	Example
Hard	$x- \rightarrow y-$	If x is blocked, y is blocked.	Cannot run deployment without artifact.
Soft	$x- \rightarrow y0$	If x is blocked, y waits.	Report can wait for optional reviewer.
Redundant	$(x1 \text{ or } x2) \rightarrow y$	y blocks only if all viable paths fail.	Failover service.

10. UOR Relational Instantiation

The UOR-aligned form treats relations as primary. Obstruction can attach to a relation independently of whether either relatum exists. This is stronger than object-only modeling.

$$\text{rel}(a,b) \in G$$

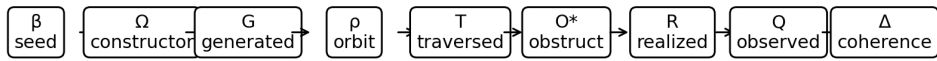
$$\tau(\text{rel}(a,b)) \in \{+,0,-\}$$

Relation status	Meaning	Example
$\text{rel}(a,b)+$	Relation is active/authorized/realized.	Person A may access resource B.
$\text{rel}(a,b)0$	Relation is latent/pending/unresolved.	Access request is under review.
$\text{rel}(a,b)-$	Relation is obstructed/denied/blocked.	Access relation denied by policy.

This is the main UOR value: existence of a and b does not determine the status of $\text{rel}(a,b)$. Authorization, privacy, delegation, memory, and governance attach to relations.

11. UAR-Agnostic Operational Mapping

ARCH-1R/T v3	Generic system term	UAR-like instantiation	Governance instantiation
β	Initial condition	Runtime seed state	Proposal submission.
Ω	Constructor	Action algebra	Policy-defined process.
G	Generated possibilities	Possible runtime states	Possible decision states.
ρ	Engine orbit	Event loop / controller walk	Review-routing process.
T	Traversed states	Explored runtime paths	Actual review path.
O*	Effective obstruction	Failures + dependency closure	Policy/resource/authority blockers.
τ	Status classifier	Runnable/waiting/blocked	Approved/pending/denied.
Q	Observation	Telemetry/replay record	Actual outcome.



12. Worked Examples

12.1 Example A: Governance Approval Flow

Scenario: a proposal P requires sponsor review, budget authority, and final approval.

β = proposal P submitted
 Ω = process constructor: attach reviewer, attach budget, attach approval route
 G = all structurally possible approval paths
 ρ = review orbit through G
 T = actual path: sponsor_review $\rightarrow$ budget_check $\rightarrow$ final_decision

Ternary status:

State **τ Reason**

Sponsor review + Sponsor has accepted review.

Budget check 0 Awaiting budget office response.

Final approval 0 Latent until budget check resolves.

If budget authority denies the request, the budget state becomes negative. If final approval hard-depends on budget, final approval also becomes negative. If final approval can proceed with conditional funding, final approval may remain neutral.

hard: budget- $\rightarrow$ approval-

soft: budget- $\rightarrow$ approval0

12.2 Example B: Runtime Dependency Cascade

Scenario: a runtime action requires service A, cache B, and optional analytics C.

β = clean runtime state
 Ω = action constructor
 ρ = controller/event orbit
 D = dependencies: A $\rightarrow$ action, B $\rightarrow$ action, C $\rightarrow$ action (soft)

Component Failure Propagation **Result**

Service A A- Hard dependency action-

Cache B B- Hard dependency action-

Analytics C C- Soft dependency action0 or action+ with degraded telemetry

The example shows why v3 matters: not every missing component should be treated as a fatal obstruction. Soft failures propagate latency or degradation, not necessarily blockage.

12.3 Example C: Terminal-Loss / Amnesia

Scenario: a system is asked to reconstruct a prior state, but the terminal anchor has been deliberately or structurally removed.

β = original identity / seed / memory anchor
 If β exists:
 $G := \text{Closure}\Omega(\beta)$
 If β is withdrawn:
 $\beta = \emptyset$
 $G = T = R = \emptyset$

This is not path failure. There is no path to classify as blocked because the generated space does not form. The appropriate class is terminal-obstruction, not downstream obstruction.

$O\beta \neq OT$

12.4 Example D: UOR Relation Authorization

Scenario: user a and record b both exist. The question is whether the relation $\text{access}(a,b)$ is active, latent, or obstructed.

Object existence Relation τ Explanation

a exists; b exists $\text{access}(a,b) +$ Authorized relation exists.

a exists; b exists $\text{access}(a,b) 0$ Request pending review.

a exists; b exists $\text{access}(a,b) -$ Policy denies relation.

This shows why relation-first modeling is more expressive than object-only modeling. Both objects may exist while the relation between them is obstructed.

12.5 Example E: Open-Orbit Latency

Scenario: the ρ -walk has entered a state but has not closed into a realized path. This is a candidate algebraic reading of latency.

```
G = {a, b, c, d}
ρ-walk observed: a → b → c
closure condition requires return, completion, or proof of d
state d is not blocked; it is unclosed
τ(d) = 0
```

This remains a research hypothesis, but it gives a testable target: define closure conditions for ρ -orbits and determine whether unclosed orbits correspond to latent states.

13. Implementation Model

A simple implementation can treat the framework as a graph plus classifier plus obstruction closure.

```
class ArchState:
    beta: Seed | None
    generator: Constructor
    rho: Engine
    graph: DependencyGraph
    observations: set[ObservedState]

procedure evaluate(system):
    if system.beta is None:
        return Empty(G=None, T=None, R=None, reason='terminal-obstruction')

    G = closure(system.generator, system.beta)
    T = orbit(system.rho, G)
    O = detect_obstructions(T)
    O_star = dependency_closure(system.graph, O)
    R = T - O_star
    tau = classify_ternary(T, O_star)
    delta = distance(R, system.observations)
    return Report(G, T, O_star, R, tau, delta)
```

Minimal data structures:

Data structure	Required fields
State node	id, type, status, cost(optional), observation(optional)
Relation edge	source, target, dependency_type: hard/soft/redundant
Obstruction record class: $O\beta/O\Omega/O\rho/OT/OD$	target, cause, timestamp
Orbit record	ρ -step, predecessor, successor, closure_status
Validation record	predicted R, observed Q, Δ metric

14. Falsification and Review Criteria

The framework should remain falsifiable. A model instance fails when predicted realization diverges from observation beyond an agreed threshold and the divergence cannot be explained by missing observation or metric error.

If $\Delta > \varepsilon$ consistently, revise β , Ω , ρ , O^* , τ , or d .

Claim	How to test	Failure mode
ρ as orbit improves explanation	Compare orbit-based T vs scheduler-based T on the same graph.	No diagnostic gain or worse prediction.
Terminal-obstruction is distinct	Create cases with β -loss and path-block; compare outcomes.	β -loss can be fully represented as ordinary OT.
$0 \neq$ - improves operations	Measure false-failure classification before/after ternary model.	No reduction in false failure reports.
Hard/soft propagation is useful	Simulate dependency cascades with typed edges.	Typed propagation does not improve prediction of actual outcomes.

15. Open Questions

- Can latency be formally characterized as an open or unclosed ρ -orbit?
- Which obstruction classes are primitive and which are reducible?
- How should cost, risk, and probability be layered without breaking the original generator?
- Can terminal-loss be detected empirically, or only inferred from non-reconstructability?
- What are the minimal simulations needed to distinguish path obstruction from terminal obstruction?
- How does dual- ρ composition behave in multi-agent or distributed traversal?

16. Compact Seals

16.1 Formal Seal

$\beta \cup \Omega \Rightarrow G$
 $G \bowtie \rho \Rightarrow T$
 $T \otimes O^* \Rightarrow R$
 $T \xrightarrow{\tau} \{+, 0, -\}$
 $R \ominus Q \Rightarrow \Delta$

16.2 Full One-Line v3

$\beta \cup \Omega \Rightarrow G \bowtie \rho \Rightarrow T \otimes O^* \Rightarrow R \ominus Q \Rightarrow \Delta$

16.3 Blade / Glyph Seal

$\beta \cup \Omega \Rightarrow G \bowtie (\text{blade} \perp \text{glyph}) \Rightarrow T \otimes O^* \Rightarrow R \ominus Q \Rightarrow \Delta$

16.4 Terminal Seal

$\odot \beta = \emptyset \Rightarrow G = T = R = \emptyset$

Final compression: the canon names what can be made. R/T names what was reached. The walk is the engine read across. The seed withdrawn is not blocked; it is gone.

Appendix A — Established vs Proposed

Item	Status	Reason
Recursive closure schema	Established formal core	Standard fixed-point/generative form; preserved from original.
NAND universality	Established in Boolean domain	Classical functional completeness.
EML and Blade instantiations	Hypotheses / domain constructions	Useful but require further proof or simulation.
R/T as operational dual	Strong framing claim	Consistent with lineage and external review.
ρ as orbit	Strong formal revision	Improves alignment with canonical ρ .
Terminal-obstruction	Strong proposed primitive	Identifies a distinct pre-generative failure class.
Open-orbit latency	Speculative research target	Interesting but not yet constructed.

Appendix B — Revision Checklist for v2 to v3

1. Replace generic traversal language with $\text{orbit}(\rho, G)$.
2. Add terminal-obstruction / β -loss to primitive obstruction classes.
3. Move or clarify any statement implying O only acts downstream of T.
4. Separate Aletheia as medium at ρ -layer and Lethe as substrate at β -loss layer.
5. Retain ternary reachability as classifier after traversal.
6. Add hard vs soft dependency propagation rules.
7. Include UOR relation-first examples and UAR-agnostic examples.

Draft Review v3.0 · Prepared for John Haines / Xarvus / Chaos Rider